

Jenkins

1. Install Jenkins and run Jenkins on port number 8081.

Objective

Install Jenkins on an AWS Amazon Linux EC2 instance and configure Jenkins to listen on:
TCP 8081

Purpose

Jenkins normally uses port 8080.
we are t=changing to port 8081

Steps:

installed through jenkins official Site:

Step 1: Long Term Support release

```
sudo wget -O /etc/yum.repos.d/jenkins.repo \
    https://pkg.jenkins.io/rpm-stable/jenkins.repo
sudo yum upgrade
# Add required dependencies for the jenkins package
sudo yum install fontconfig java-21-openjdk
sudo yum install jenkins
sudo systemctl daemon-reload
```

Step 2 :Java

```
sudo yum install -y java-21
```

Step 3: Port change:

```
vi systemctl edit jenkins
                                     (or)
vi /etc/systemd/system/jenkins.service.d/override.conf
```

Step 4:

```
[Service]
Environment="JENKINS_PORT=8081"
```

```
sudo systemctl daemon-reload
sudo systemctl restart Jenkins
sudo systemctl status jenkins
```

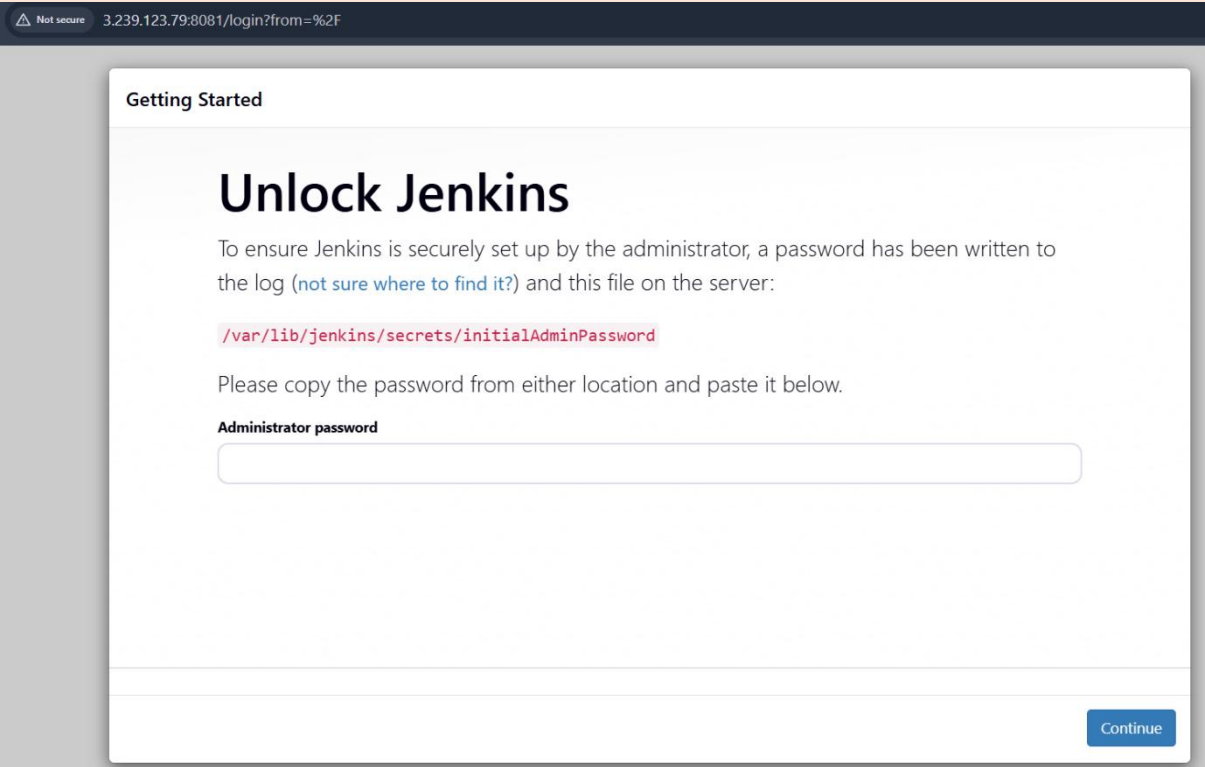
Step 5:

To verify port number:

```
sudo ss -ltnp | grep 8081
```

Jenkins

Implementation:



Getting Started

Getting Started

✓ Folders	✓ OWASP Markup Formatter	✓ Build Timeout	⌚ Credentials Binding	** commons-lang3 v3.x Jenkins API
⌚ Timestampers	⌚ Workspace Cleanup	⌚ Ant	⌚ Gradle	** Ionicons API
⌚ Pipeline	⌚ GitHub Branch Source	⌚ Pipeline: GitHub Groovy Libraries	⌚ Pipeline Graph View	Folders
⌚ Git	⌚ SSH Build Agents	⌚ Matrix Authorization Strategy	⌚ LDAP	OWASP Markup Formatter
⌚ Email Extension	⌚ Mailer	⌚ Dark Theme		** ASM API
				** JSON Path API
				** Struts
				** Pipeline: Step API
				** commons-text API
				** Token Macro
				Build Timeout
				** bouncycastle API
				** Credentials
				** Plain Credentials
				** Variant
				** SSH Credentials
				** - required dependency

Jenkins 2.568.3

Conclusion

Jenkins is installed on the Amazon Linux EC2 instance, configured as a systemd service, enabled for automatic startup, and exposed through TCP port 8081.

Jenkins

2. Secure Jenkins server

Objective

Secure Jenkins by enabling authenticated access and implementing controlled authorization.

Purpose

A Jenkins server should not allow anonymous users to administer Jenkins or execute arbitrary builds.

Steps:

Step 1: Log in as the Jenkins administrator.

Step 2: Navigate to: Dashboard

→ Manage Jenkins → Security

Step 3: Under **Security Realm**, select:

Jenkins' own user database

Step 4: Do not enable public self-registration for this lab.

Step 5: Under **Authorization**, select: Matrix-based security

Step 6: Add the administrator user.

Give the administrator: Overall → Administer

Step 7: Check the Anonymous permissions.

Ensure Anonymous does **not** have:

Overall → Administer

Step 8: Click:

Save

Step 9: Verify that the administrator can still access:

Manage Jenkins

Implementation:

```
[ec2-user@ip-172-31-1-225 ~]$ sudo su
[root@ip-172-31-1-225 jenkins]# ll
total 76
-rw-r--r--. 1 jenkins jenkins 2485 Sep 28 11:27 config.xml
-rw-r--r--. 1 jenkins jenkins 156 Sep 28 10:44 hudson.model.UpdateCenter.xml
-rw-r--r--. 1 jenkins jenkins 370 Sep 28 10:52 hudson.plugins.git.GitTool.xml
-rw-r--r--. 1 jenkins jenkins 1680 Sep 28 10:52 identity.key.enc
-rw-r--r--. 1 jenkins jenkins 7 Sep 28 10:54 jenkins.install.InstallUtil.lastExecVersion
-rw-r--r--. 1 jenkins jenkins 7 Sep 28 10:54 jenkins.install.UpgradeWizard.state
-rw-r--r--. 1 jenkins jenkins 182 Sep 28 10:54 jenkins.model.JenkinsLocationConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 171 Sep 28 10:15 jenkins.telemetry.Correlator.xml
drwxr-xr-x. 2 jenkins jenkins 6 Sep 28 11:27 jobs
-rw-r--r--. 1 jenkins jenkins 1037 Sep 28 10:44 nodeMonitors.xml
drwxr-xr-x. 94 jenkins jenkins 16384 Sep 28 10:52 plugins
-rw-r--r--. 1 jenkins jenkins 258 Sep 28 10:43 queue.xml.bak
-rw-r--r--. 1 jenkins jenkins 64 Sep 28 10:15 secret.key
-rw-r--r--. 1 jenkins jenkins 0 Sep 28 10:15 secret.key.not-so-secret
drwx-r--r--. 2 jenkins jenkins 16384 Sep 28 11:21 secrets
drwxr-xr-x. 2 jenkins jenkins 149 Sep 28 10:52 updates
drwxr-xr-x. 2 jenkins jenkins 24 Sep 28 10:15 userContent
drwxr-xr-x. 3 jenkins jenkins 84 Sep 28 10:15 users
[root@ip-172-31-1-225 jenkins]# cd secrets/
[root@ip-172-31-1-225 secrets]# ll
total 28
-rw-r--r--. 1 jenkins jenkins 48 Sep 28 10:15 hudson.model.User.DIRNAMES
-rw-r--r--. 1 jenkins jenkins 272 Sep 28 11:21 hudson.util.Secret
-rw-r--r--. 1 jenkins jenkins 33 Sep 28 10:15 initialAdminPassword
-rw-r--r--. 1 jenkins jenkins 32 Sep 28 10:15 jenkins.model.Jenkins.crumbSalt
-rw-r--r--. 1 jenkins jenkins 48 Sep 28 10:23 jenkins.security.csp.ReportingContext.key
-rw-r--r--. 1 jenkins jenkins 256 Sep 28 10:15 master.key
-rw-r--r--. 1 jenkins jenkins 272 Sep 28 10:52 org.jenkinsci.main.modules.instance_identity.InstanceIdentity.KEY
[root@ip-172-31-1-225 secrets]# cat initialAdminPassword
1afa50b961eb47dd959eca9b9ccff143
[root@ip-172-31-1-225 secrets]# rm -rf initialAdminPassword
[root@ip-172-31-1-225 secrets]# |
```

Jenkins

The image shows two screenshots of the Jenkins web interface. The top screenshot is the login page, titled "Sign in to Jenkins". It features a large cartoon illustration of a man in a suit on the left. On the right, there are input fields for "Username" (containing "admin") and "Password" (containing masked characters). Below the password field is a checkbox for "Keep me signed in" and a blue "Sign in" button. The browser's address bar shows the URL "3.239.123.79:8081/login?from=%2Fuser%2Fadmin%2F".

The bottom screenshot is the "configureSecurity/" page. It has a breadcrumb trail: "Jenkins / Manage Jenkins / Security". The "Security Realm" is set to "Jenkins' own user database". There is an unchecked checkbox for "Allow users to sign up". The "Authorization" is set to "Matrix-based security". Below this is a table of users and groups:

Filter users and groups		Add user	Add group
admin	Overall: Administrator		
Anonymous	No permissions granted		
Authenticated Users	No permissions granted		

Below the table is the "Markup Formatter" section, set to "Plain text". A note states: "Shows descriptions mostly as written. HTML unsafe characters like < and & are escaped to their respective character entities, and line breaks are converted to their HTML equivalent." At the bottom, there is an "Agents" section and "Save" and "Apply" buttons.

Conclusion:

Jenkins authentication and authorization are now configured so users must authenticate and permissions can be granted individually rather than giving every user administrative control.

Jenkins

3. Create users called DevOps, Testing in Jenkins with Limited access.

Objective

To create two Jenkins users named **DevOps** and **Testing** and provide them with limited permissions.

Purpose

This demonstrates the **principle of least privilege**, where users receive only the permissions required to perform their assigned activities.

Step 1: Create DevOps user

Dashboard → Manage Jenkins → Users → Create User

Username: DevOps

Password: <strong-password>

Name: DevOps

Email: your-email@example.com

Click Create User.

Step 2: Create Testing user

Dashboard → Manage Jenkins → Users → Create User

Username: Testing

Password: <strong-password>

Name: DevOps

Email: your-email@example.com

Configure permissions

Manage Jenkins → Security → Authorization → Matrix-based security

Add: DevOps → Overall → Read

Add: Testing → Overall → Read

Implementation:

The image shows two screenshots of the Jenkins web interface. The top screenshot displays the 'Users' management page, showing a table with three users: 'admin', 'Devops', and 'Testing'. The bottom screenshot shows the profile page for the 'Testing' user, displaying a sidebar with navigation options like Profile, Builds, My Views, Account, Appearance, Preferences, Security, Experiments, and Credentials.

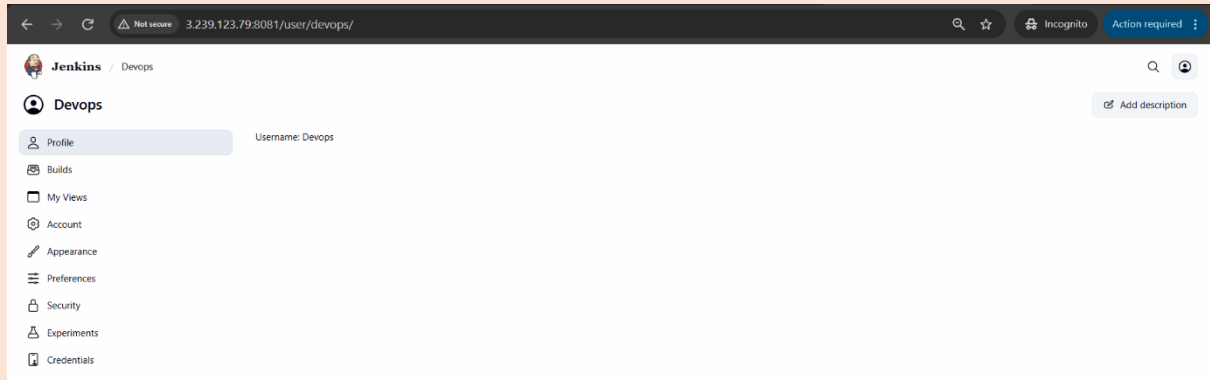
Username	Full name	Actions
admin	admin	[Settings]
Devops	Devops	[Settings] [Delete]
Testing	Testing	[Settings] [Delete]

Testing

Username: Testing

- Profile
- Builds
- My Views
- Account
- Appearance
- Preferences
- Security
- Experiments
- Credentials

Jenkins



Conclusion

The **DevOps** and **Testing** users were successfully created with restricted permissions. This demonstrates Jenkins user management, Matrix-based authorization, and least-privilege security.

4. Configure labels and restrict the jobs to execute based on label only.

Objective

To configure a Jenkins node label and restrict Jenkins jobs so that they execute only on a node with the specified label.

Purpose

Jenkins labels are used to control where jobs execute. For example, a job configured with the devops label runs on a Jenkins node that has that label.

Steps:

Step 1: Create the Jenkins job.

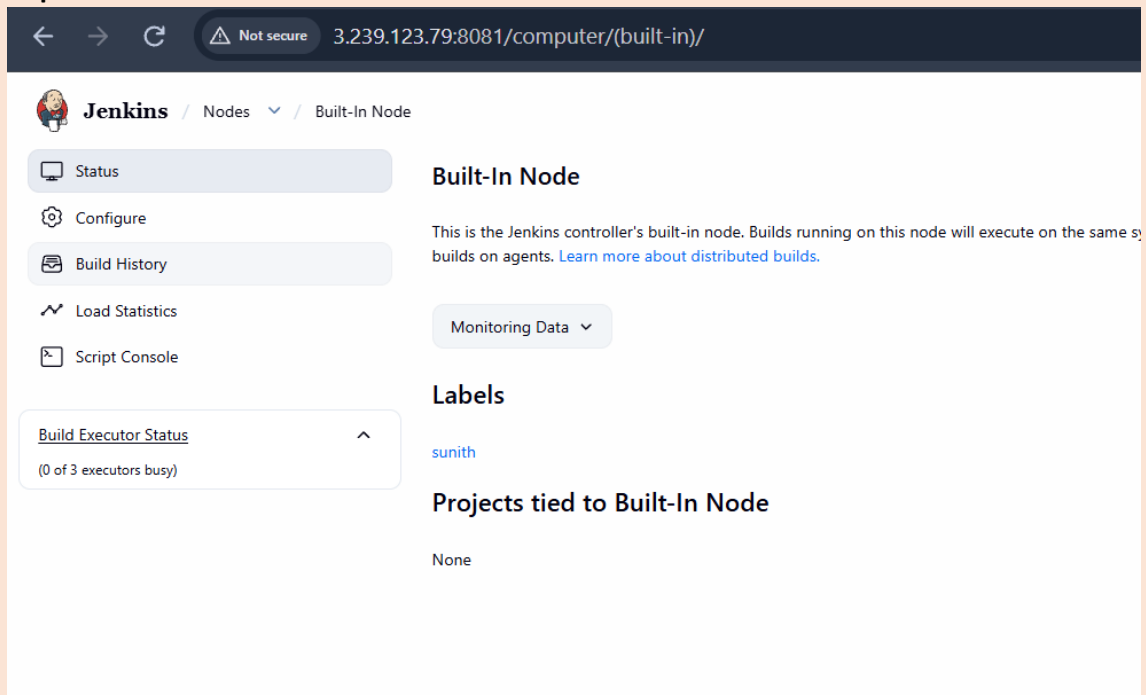
Step 2: Restrict execution to:Sunith

Step 3: Configure Git. With link and branch

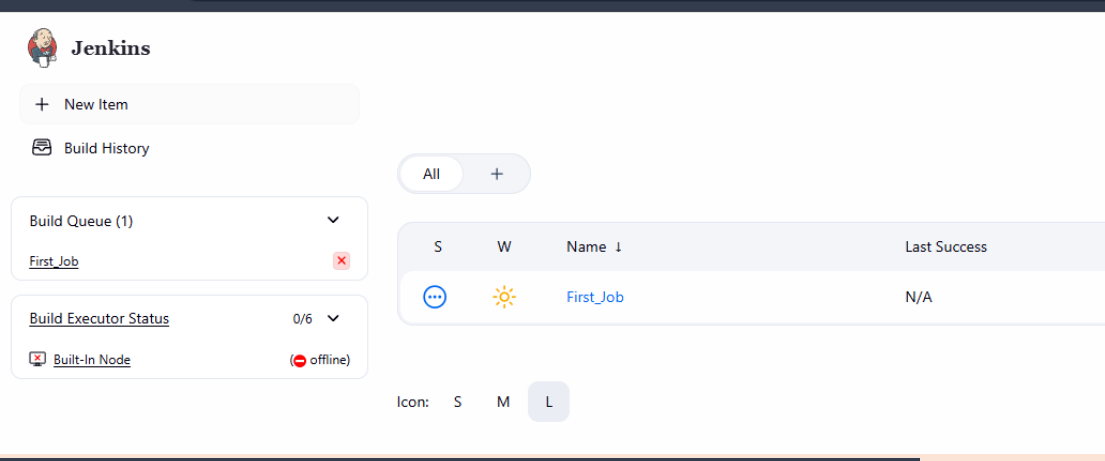
Step 4: Save and click

Step 5: Build Now

Implementation:

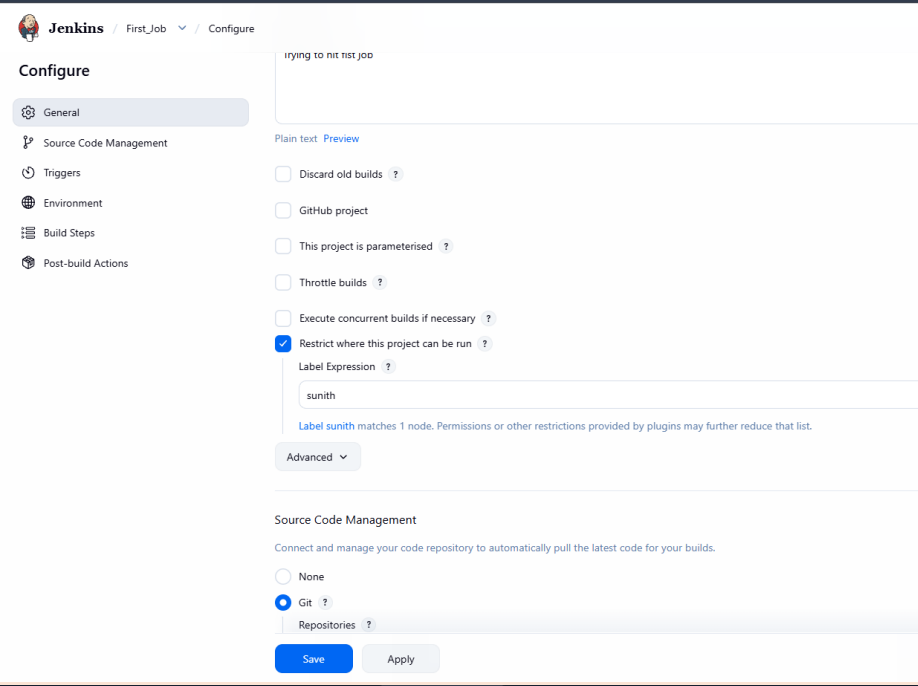


Jenkins



The Jenkins Dashboard shows the main interface with a 'New Item' button, 'Build History' link, and a table of build queues. The 'Build Queue (1)' section shows a job named 'First_Job' with a status icon. The 'Build Executor Status' section shows 'Built-In Node' as 'offline'. A table at the bottom shows build status (S, W, Name, Last Success) for 'First_Job' with a status of 'N/A'.

S	W	Name	Last Success
...	☀	First_Job	N/A



The Jenkins Job Configuration page for 'First_Job' is shown. The 'General' tab is selected. The 'Restrict where this project can be run' checkbox is checked, and the 'Label Expression' is set to 'sunith'. The 'Source Code Management' section shows 'Git' as the selected provider. The 'Save' button is highlighted.

Configure

General

Source Code Management

Triggers

Environment

Build Steps

Post-build Actions

Discard old builds ?

GitHub project

This project is parameterised ?

Throttle builds ?

Execute concurrent builds if necessary ?

☒ Restrict where this project can be run ?

Label Expression ?

sunith

Label sunith matches 1 node. Permissions or other restrictions provided by plugins may further reduce that list.

Advanced

Source Code Management

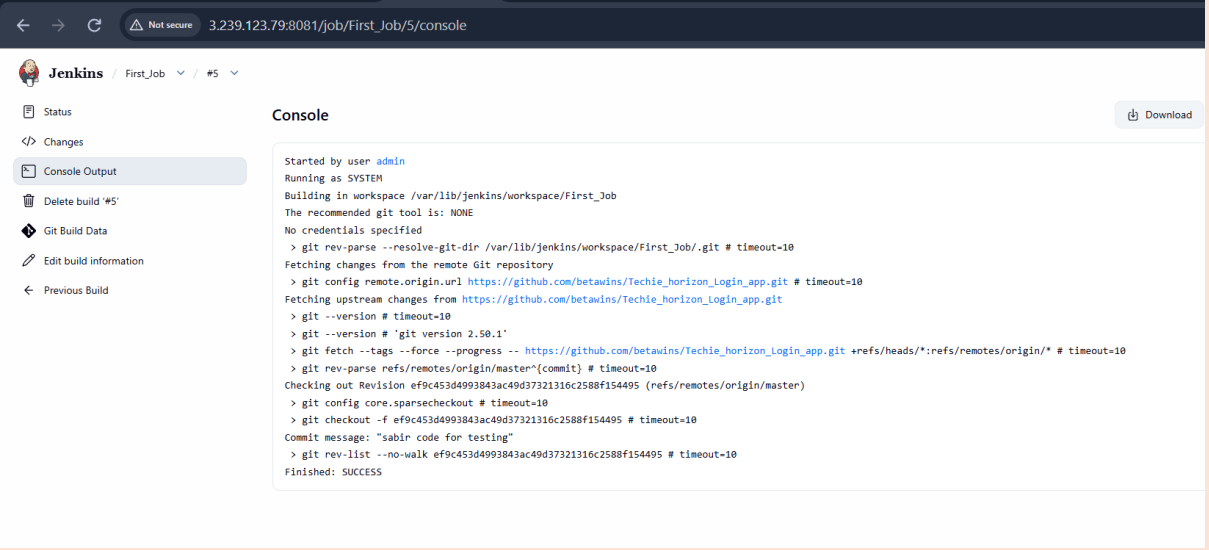
Connect and manage your code repository to automatically pull the latest code for your builds.

None

☒ Git ?

Repositories ?

Save Apply



The Jenkins Console Output page for 'First_Job' is shown. The console output displays the build process, including the user 'admin', the workspace path, and the git commands executed. The build is successful.

Started by user admin

Running as SYSTEM

Building in workspace /var/lib/jenkins/workspace/First_Job

The recommended git tool is: NONE

No credentials specified

> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/First_Job/.git # timeout=10

Fetching changes from the remote Git repository

> git config remote.origin.url https://github.com/betawins/Techie_horizon_Login_app.git # timeout=10

Fetching upstream changes from https://github.com/betawins/Techie_horizon_Login_app.git

> git --version # timeout=10

> git --version # 'git version 2.50.1'

> git fetch --tags --force --progress -- https://github.com/betawins/Techie_horizon_Login_app.git +refs/heads/*:refs/remotes/origin/* # timeout=10

> git rev-parse refs/remotes/origin/master^{commit} # timeout=10

Checking out Revision ef9c453d4993843ac49d37321316c2588f154495 (refs/remotes/origin/master)

> git config core.sparsecheckout # timeout=10

> git checkout -f ef9c453d4993843ac49d37321316c2588f154495 # timeout=10

Commit message: "sabin code for testing"

> git rev-list --no-walk ef9c453d4993843ac49d37321316c2588f154495 # timeout=10

Finished: SUCCESS

Conclusion

The Jenkins node was assigned the devops label and the job was restricted to that label. The successful build confirms that Jenkins executed the job on the required labeled node

Jenkins

5. Create Three sample jobs using the below URL.

Objective

To create three Jenkins jobs using the specified public GitHub repository:

- **Source Code:** https://github.com/betawins/Techie_horizon_Login_app.git

The repository uses the master branch and contains a Maven/Spring MVC application with a pom.xml.

Implementation:

steps:

create new items

enter the name:

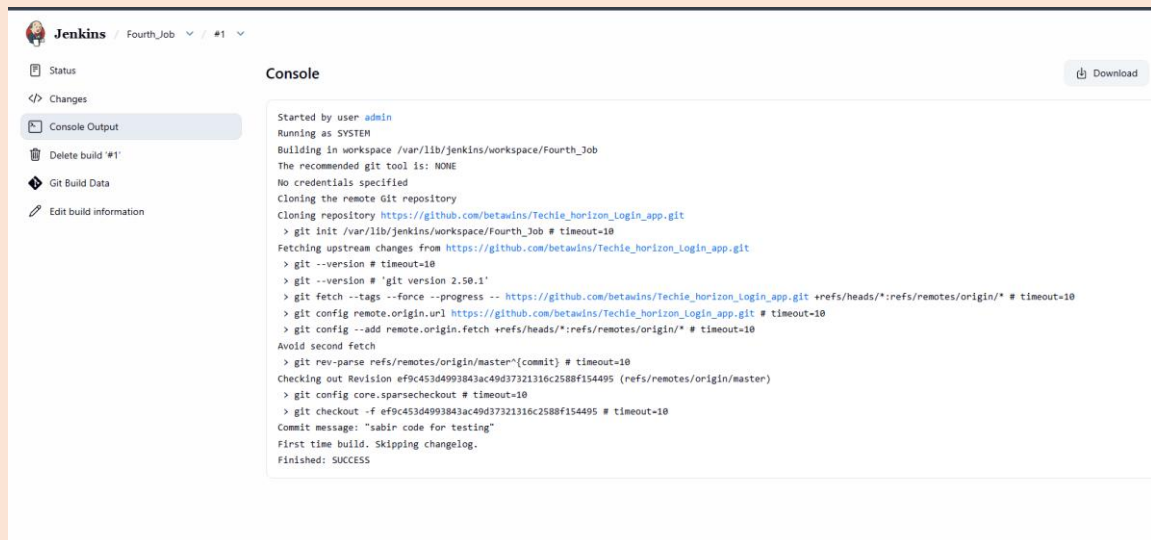
select the free style

The image displays two screenshots of the Jenkins web interface, specifically the console output for two different jobs. Both screenshots show the Jenkins logo and navigation tabs (Status, Changes, Console Output, Delete build, Git Build Data, Edit build information). The top screenshot is for 'Second_Job' and shows the console output for build #1. The bottom screenshot is for 'Third_Job' and shows the console output for build #2. Both jobs are configured to use the GitHub repository 'https://github.com/betawins/Techie_horizon_Login_app.git' and the 'master' branch. The console output for both jobs shows the same sequence of commands: 'git init', 'git fetch', 'git config', 'git checkout', and 'git commit'. The output for 'Second_Job' shows a successful build with the message 'Finished: SUCCESS'. The output for 'Third_Job' shows a successful build with the message 'Finished: SUCCESS'.

```
Started by user admin
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/Second_Job
The recommended git tool is: NONE
No credentials specified
Cloning the remote git repository
Cloning repository https://github.com/betawins/Techie_horizon_Login_app.git
> git init /var/lib/jenkins/workspace/Second_Job # timeout=10
Fetching upstream changes from https://github.com/betawins/Techie_horizon_Login_app.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/betawins/Techie_horizon_Login_app.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git config remote.origin.url https://github.com/betawins/Techie_horizon_Login_app.git # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git rev-parse refs/remotes/origin/master^{commit} # timeout=10
Checking out Revision ef9c453d4993843ac49d37321316c2588f154495 (refs/remotes/origin/master)
> git config core.sparsecheckout # timeout=10
> git checkout -f ef9c453d4993843ac49d37321316c2588f154495 # timeout=10
Commit message: "sabin code for testing"
First time build. Skipping changelog.
Finished: SUCCESS
```

```
Started by user admin
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/Third_Job
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/Third_Job/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/betawins/Techie_horizon_Login_app.git # timeout=10
Fetching upstream changes from https://github.com/betawins/Techie_horizon_Login_app.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/betawins/Techie_horizon_Login_app.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/master^{commit} # timeout=10
Checking out Revision ef9c453d4993843ac49d37321316c2588f154495 (refs/remotes/origin/master)
> git config core.sparsecheckout # timeout=10
> git checkout -f ef9c453d4993843ac49d37321316c2588f154495 # timeout=10
Commit message: "sabin code for testing"
> git rev-list --no-walk ef9c453d4993843ac49d37321316c2588f154495 # timeout=10
Finished: SUCCESS
```

Jenkins



Conclusion

Successfully implemented the task

6. Create one Jenkins job using GitHub Private repository.

Objective

To configure Jenkins to securely access and build a private GitHub repository using Jenkins-managed credentials.

Purpose

Unlike a public repository, a private GitHub repository requires authentication. Jenkins credentials allow authentication information to be stored securely rather than exposing tokens directly in Git URLs or scripts.

Step 1: Created a private GitHub repository.

Step 2: Create GitHub authentication.

GitHub Username + Personal Access Token

Step 3: Create a GitHub Personal Access Token with the required repository access

Step 4: Add the credential to Jenkins.

Go to: Dashboard

- Manage Jenkins
- Credentials
- Global credentials
- Add Credentials

Enter:

Username: YOUR_GITHUB_USERNAME

Password: YOUR_GITHUB_TOKEN

ID: github-private-repo

Description: GitHub credential for private repository

Step 5: Create the Jenkins job.

Step 6: Restrict execution to:Sunith

Step 7: Configure Git.

Step 9: Save and click:

Jenkins

Implementation:

The image shows two screenshots from the Jenkins web interface. The top screenshot displays the console output for a build named 'private_repo' (build #2). The output shows the build process starting with user 'admin', running as 'SYSTEM', and cloning a private GitHub repository using a Jenkins-managed credential. The build is successful.

```
Started by user admin
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/private_repo
The recommended git tool is: NONE
using credential dfghjkl
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/private_repo/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/Sunith3210/jenkins_test.git # timeout=10
Fetching upstream changes from https://github.com/Sunith3210/jenkins_test.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
using GIT_ASKPASS to set credentials
> git fetch --tags --force --progress -- https://github.com/Sunith3210/jenkins_test.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main{commit} # timeout=10
Checking out Revision e2e0b1306e18310ad19adc58c7c7737dada66420 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f e2e0b1306e18310ad19adc58c7c7737dada66420 # timeout=10
Commit message: "Initial commit"
First time build. Skipping changelog.
[private_repo] $ /bin/sh -xe /tmp/jenkins4736346447124489614.sh
+ echo 'Private repo was successfully cloned'
Private repo was successfully cloned
Finished: SUCCESS
```

The bottom screenshot shows the 'Configure' page for a job named 'First_Job'. The 'General' tab is selected. Under 'Restrict where this project can be run', the checkbox 'Restrict where this project can be run' is checked, and the 'Label Expression' is set to 'sunith'. The 'Source Code Management' section shows 'Git' as the selected provider.

Configure

General

Source Code Management

Triggers

Environment

Build Steps

Post-build Actions

Trying to nit test job

Plain text [Preview](#)

☐ Discard old builds ?

☐ GitHub project

☐ This project is parameterised ?

☐ Throttle builds ?

☐ Execute concurrent builds if necessary ?

☒ Restrict where this project can be run ?

Label Expression ?

sunith

Label sunith matches 1 node. Permissions or other restrictions provided by plugins may further reduce that list.

Advanced ▾

Source Code Management

Connect and manage your code repository to automatically pull the latest code for your builds.

☐ None

☒ Git ?

Repositories ?

Save Apply

Conclusion

The private GitHub repository was integrated with Jenkins using a Jenkins-managed credential. This demonstrates secure source-code authentication and avoids exposing sensitive GitHub authentication information in job configuration or scripts